



PROGRAMACION 2

TP N° 2: Herencia y Polimorfismo en Java

ALUMNO: LUIS CISNEROS

COMISIÓN: 11

1. Vehículos y herencia básica

```
public class Vehiculo {
    protected String marca;
    protected String modelo;

    public Vehiculo(String marca, String modelo) {
        this.marca = marca;
        this.modelo = modelo;
    }

    public void mostrarInfo() {
        System.out.println("Marca: " + marca + ", Modelo: " + modelo);
    }
}

// Subclase Auto
class Auto extends Vehiculo {
    private int cantidadPuertas;
```

```

public Auto(String marca, String modelo, int cantidadPuertas) {
    super(marca, modelo);
    this.cantidadPuertas = cantidadPuertas;
}

@Override
public void mostrarInfo() {
    super.mostrarInfo();
    System.out.println("Cantidad de puertas: " + cantidadPuertas);
}
}

```

```

public class EjecucionKata1 extends Ejecutable {
    // Clase principal para prueba

    public void execute() {
        Vehiculo miVehiculo = new Vehiculo("Ford", "Fiesta");
        Auto miAuto = new Auto("Toyota", "Corolla", 4);
        miAuto.mostrarInfo();
    }
}

```

2. Figuras geométricas y métodos abstractos

```

// Clase abstracta Figura
abstract class Figura {
    protected String nombre;

    public Figura(String nombre) {
        this.nombre = nombre;
    }

    public abstract double calcularArea();

    public void mostrarInfo() {
        System.out.println("Figura: " + nombre + ", Área: " + calcularArea());
    }
}

```

```

// Subclase Círculo
class Circulo extends Figura {
    private double radio;

    public Circulo(String nombre, double radio) {

```

```

        super(nombre);
        this.radio = radio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }
}

```

```

// Subclase Rectángulo
class Rectangulo extends Figura {
    private double base;
    private double altura;

    public Rectangulo(String nombre, double base, double altura) {
        super(nombre);
        this.base = base;
        this.altura = altura;
    }

    @Override
    public double calcularArea() {
        return base * altura;
    }
}

```

```

public class EjecucionKata2 extends Ejecutable {
    public void execute() {
        Figura[] figuras = new Figura[3];
        figuras[0] = new Circulo("Círculo pequeño", 5.0);
        figuras[1] = new Rectangulo("Rectángulo grande", 10.0, 6.0);
        figuras[2] = new Circulo("Círculo mediano", 7.5);

        // Polimorfismo: llamada al método calcularArea() según el tipo real
        for (Figura figura : figuras) {
            figura.mostrarInfo();
        }
    }
}

```

3. Animales y comportamiento sobrescrito

```
class Animal {
    protected String nombre;
    protected String especie;

    public Animal(String nombre, String especie) {
        this.nombre = nombre;
        this.especie = especie;
    }

    public void hacerSonido() {
        System.out.println("El animal hace un sonido genérico");
    }

    public void describirAnimal() {
        System.out.println("Soy un " + especie + " llamado " + nombre);
    }
}
```

```
// Subclase Gato
class Gato extends Animal {
    public Gato(String nombre) {
        super(nombre, "Gato");
    }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + " dice: ¡Miau miau!");
    }
}
```

```
// Subclase Perro
class Perro extends Animal {
    public Perro(String nombre) {
        super(nombre, "Perro");
    }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + " dice: ¡Guau guau!");
    }
}
```

```
// Subclase Vaca
class Vaca extends Animal {
    public Vaca(String nombre) {
        super(nombre, "Vaca");
    }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + " dice: ¡Muuuu!");
    }
}
```

```
public class EjecucionKata3 extends Ejecutable {
    public void execute() {
        List<Animal> animales = new ArrayList<>();
        animales.add(new Perro("Rex"));
        animales.add(new Gato("Misi"));
        animales.add(new Vaca("Lola"));
        animales.add(new Perro("Fido"));
        animales.add(new Gato("Garfield"));

        // Polimorfismo: llamada al método hacerSonido() según el tipo real
        for (Animal animal : animales) {
            animal.describirAnimal();
            animal.hacerSonido();
            System.out.println("---");
        }
    }
}
```

4. Empleados y polimorfismo

```
// Clase abstracta Empleado
abstract class Empleado {
    protected String nombre;
    protected double salarioBase;

    public Empleado(String nombre, double salarioBase) {
        this.nombre = nombre;
        this.salarioBase = salarioBase;
    }
}
```

```

    public abstract double calcularSueldo();

    public String getNombre() {
        return nombre;
    }
}

/ Subclase EmpleadoPlanta
class EmpleadoPlanta extends Empleado {
    private double bonoAntiguedad;

    public EmpleadoPlanta(String nombre, double salarioBase, double
bonoAntiguedad) {
        super(nombre, salarioBase);
        this.bonoAntiguedad = bonoAntiguedad;
    }

    @Override
    public double calcularSueldo() {
        return salarioBase + bonoAntiguedad;
    }
}

// Subclase EmpleadoTemporal
class EmpleadoTemporal extends Empleado {
    private double horasExtras;
    private double tarifaHoraExtra;

    public EmpleadoTemporal(String nombre, double salarioBase, double
horasExtras, double tarifaHoraExtra) {
        super(nombre, salarioBase);
        this.horasExtras = horasExtras;
        this.tarifaHoraExtra = tarifaHoraExtra;
    }

    @Override
    public double calcularSueldo() {
        return salarioBase + (horasExtras * tarifaHoraExtra);
    }
}

```